How we got here

Every question you asked, what it changed, and what it yielded

SRON COGNITO post-doc application

Vikrant Tomar · August 2026

Why this report exists.

The project moved through about thirty exchanges over two days. Several of your questions changed the science, not just the code — three of them caught errors that would have invalidated the result, and one led directly to the strongest finding in the project.

This is the record: what you asked, what the answer was, what changed as a result, and whether it was worth it.

AT A GLANCE

The five questions that changed the outcome

Most exchanges were ordinary problem-solving. These five were not.

You asked	It uncovered
“Why is it downloading NO_2… EEException”	A date-format bug that had silently broken every nitrogen dioxide and fire lookup. Without this, the fire discriminator — your distinctive contribution — would never have existed.
“Did you make each code or are they from Schuit?”	That their code was never released, so everything here is a reimplementation. Led to PROVENANCE.md and the disclosure advice that protects you.
“I cannot see much signal at some steel sites”	That single overpasses mostly do not show a steel-plant plume — and that stacking a site's days reveals one. This became a central result.
“Check whether it is providing right coordinates”	One site 11.4 km out, and confirmation that Kosice's weak signal is real rather than a pointing error.
“What about the days when the satellite doesn't pick anything”	A 28% selection effect that biases the whole dataset toward inland dry-season scenes, and prompted the seasonal test that became the strongest control.

PHASE 1

Working out what the job needs

You asked	The answer	What changed
“Analyse this” (the SRON vacancy PDF)	The post is COGNITO — build the first global system for detecting and quantifying CO emissions from cities and steel plants using TROPOMI and machine learning. Two of the three named 'assets' are CO research and transport modelling, and you have both. The gap is that your machine learning is regression on tables, and this job is pattern recognition in pictures.	A written assessment saved to the project. Identified the biggest single opportunity: the advertisement says *apply existing* algorithms, not invent.
“How can I do that?”	Because the method is published, its training data is public, and TROPOMI CO is a few lines away in Earth Engine, a two-day demonstration is feasible. The question worth asking: does a plume detector trained on methane transfer to carbon monoxide?	A working code scaffold plus an hour-by-hour plan. Every script tested end to end on synthetic data before you touched it.
“Is the training data CH_4? How do we create it for CO?”	You never do. The network is trained on one normalised channel, so it cannot know which gas it is looking at. Strip the units and a methane plume and a CO plume are the same picture. The transfer *is* the experiment.	A figure proving it numerically: same plume, 2.8% enhancement in methane and 35.5% in CO, but both peaking at ~3.4 after normalising.

PHASE 2

Provenance, documentation, and making it usable

You asked	The answer	What changed
“Did you make each code, or are they from Schuit?”	I checked. Schuit et al. released their training data and results but **not their source code** — the paper's assets page lists data DOIs and WRF, no repository. So every line is a reimplementation from the paper's prose.	PROVENANCE.md, listing line by line what comes from the papers and what is a judgement call. Plus direct advice: read every line, change what you disagree with, and disclose the AI assistance in your README.
“Explain everything in simple language, as a PDF, with flow charts”	Produced a 15-page guide with seven diagrams, assuming no neural-network knowledge and using analogies from your own work — the network as a matched filter, training as fitting a calibration curve, the train/test split as keeping a validation cylinder separate from the calibration cylinder.	That guide. It is the document to re-read before an interview.
“train.py is taking time but nothing comes out — I cannot tell if it hung”	You were right, and it was worse than you thought: two separate stretches ran silently for minutes, including a normalisation loop hidden inside a helper.	Rewritten with per-batch progress, a whole-run time estimate from the second round, safe Ctrl-C that keeps the best model, a three-minute **--quick** mode, and output that works in notebooks and log files where the old bar was invisible.

PHASE 3

Understanding the data product

You asked	The answer	What changed
“What is the resolution of L3_CO? What if we use L2?”	The catalogue's 1113.2 m is the **grid spacing**, not the resolution. The real footprint is 7.0 by 7.0 km at nadir, 7.0 by 5.5 km since August 2019 — the grid oversamples the truth six times over. L2 gives back the quality threshold as your choice, the averaging kernel, and the detector's own geometry — where the striping artefact is actually visible.	**Caught an error in my own code comment**, which had confused the CO and NO_2 footprints. Also produced fetch_co_l2.py with a stripiness metric that cannot be computed from the gridded product at all.
“Are they L2 or L3, for CO and NO_2?”	Both are Google's 'L3', which is really gridded L2 — ESA publishes no L3 for Sentinel-5P at all. Only the MODIS fire product is a genuine L3.	A precise sentence for your README, since SRON built the operational L2 retrieval and will know the difference immediately.
“5150 orbits… explain what is written”	Each satellite image in Earth Engine is a full pole-to-pole orbit whose bounding box wraps all longitudes near the poles — so effectively every orbit of the year 'intersects' any site. That is why the yield was only about 8%.	Explained the low yield honestly, and identified the fix that later became the day-based sampling.

PHASE 4

Getting Earth Engine to work

Six exchanges, all setup, no science. Worth recording only because the failure modes are non-obvious and you will hit them again on another machine.

Symptom	Actual cause	Fix
“Where do I put the project ID?”	The **--project** flag was never the good way to do it.	**earthengine set_project** once, and it is stored permanently.
“Credentials already exist”	Authentication was done; the project was not set.	Test with one command before assuming anything is wrong.
“Not signed up for Earth Engine”	The project existed but had never been registered.	Register for noncommercial use, which since September 2025 requires an eligibility questionnaire.
“API has not been used in project”	**Enabling the API and registering the project are two different switches.** Most people expect one.	Enable the API from the link in the error.
“Project is not registered” (again)	The API was now on, but the registration questionnaire was still outstanding.	Complete the form. Three gates, cleared one at a time.

What changed in the code because of this. The script now finds your project automatically once it is stored, forces a real connection test at startup rather than trusting a lazy one that fails later, and distinguishes five failure modes — not authenticated, no project set, project not registered, expired login, network problem — each with the specific fix. All five paths were tested.

PHASE 5

Three bugs you caught by looking at the output

This phase is the reason the final result is trustworthy. Each of these was found because you looked at what came out rather than accepting the summary line.

Bug 1 — the date format

You asked: why is it downloading NO_2, and why is every one throwing an exception?

Cause: the date was being passed as `20190101`. Earth Engine needs `2019-01-01`. The CO fetch never touches the date, which is exactly why CO worked and the other two failed on every single scene.

Yielded: the fire discriminator became possible at all. Without this, the angle that is most distinctly yours would not exist in the results.

Bug 2 — three days pretending to be a year

You asked: nothing directly — you sent the output file.

Found on inspection: all 87 tiles came from 1, 2 and 3 January 2019. The code took the *first* 40 orbits, and at 14 orbits a day that is 72 hours.

Yielded: even-spread sampling. But the first fix was not enough — see below.

Bug 3 — region tied to season

Found on inspection of the second run: European sites had been sampled only in July and August, Indian sites only in March and July. Any Europe-versus-India difference in detector score would have been uninterpretable — the fatal flaw for the whole experiment, since that comparison *is* the experiment.

Cause: because every orbit matches every site, striding picked the same 40 orbit identifiers everywhere, and which of those actually covered a site depended on orbit geometry.

Yielded: day-based sampling. Choose the days first, then ask for each day's mosaic. Yield went from 4% to 72%, and every site now shares one calendar.

Run	Tiles	Days	Confound
First	87	3 — all in January	severe
Second	45	7	Europe = summer, India = spring
Third	**335**	**18, all twelve months**	**none**

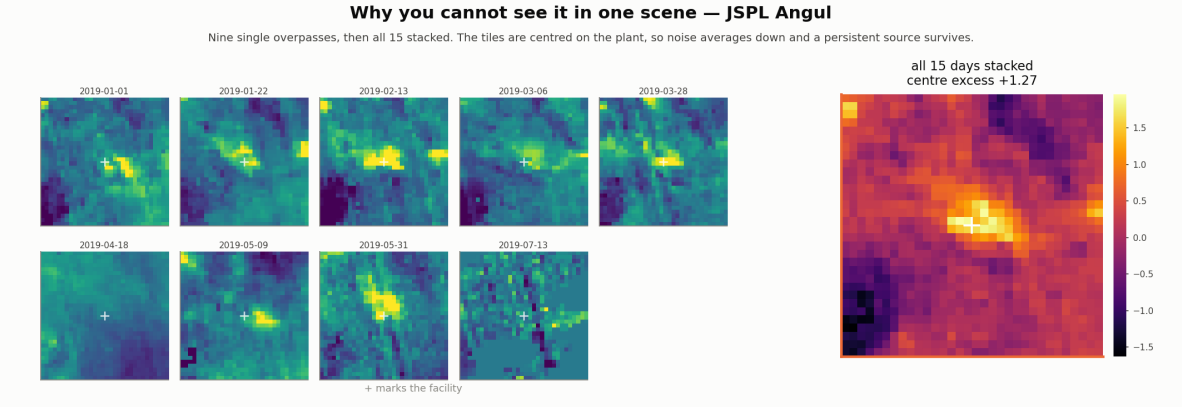
PHASE 6

The question that produced the best result

You asked: “Show me more pictures for some random days, as I cannot see much of the signal at some steel sites.”

You were right, and the number confirmed it: single-scene enhancement barely separated the groups — background 1.10, Europe 1.29, India 1.35, heavily overlapping. At 7 km resolution, one overpass usually does not show a steel-plant plume.

But every tile is centred on the facility. So stacking a site's days is exactly the oversampling technique the literature uses: noise averages down, a persistent source survives.



Group	Centre excess after stacking
Indian steel	**+0.59**
Indian cities	+0.36
European steel	+0.14
Background controls	**-0.003**

Why this matters more than the detector score. The control sits at zero, exactly as a working negative control must. And the finding itself is substantive: steel-plant CO is not reliably a single-scene detection problem — it emerges from temporal aggregation. That is precisely why Leguijt et al. use year-long inversions and wind-rotated oversampling rather than per-scene detection, and it is the sharpest statement of why porting a methane super-emitter detector to CO is not straightforward: methane super-emitters are visible per scene; steel-plant CO mostly is not.

PHASE 7

Checking the result honestly

You asked	What happened
“I also want to see where the score is zero”	The lowest-scoring scenes turned out to be almost all **striped** ones — Sahara at 0.0001. So the network *rejects* stripes rather than firing on them, which is the opposite of the risk I had flagged. I corrected that in writing.
“Check whether it is providing right coordinates”	Verified 13 of 20 steel plants against Global Energy Monitor. Twelve were inside half a satellite pixel. **JSPL Angul was 11.4 km out** — and still produced the strongest stacked signal in the dataset, meaning that plume survives a one-pixel pointing error. Also confirmed Kosice's weak signal is *real*, not a coordinate problem.
“Let's discuss — what does it infer?”	Testing the figure's own caption showed I had **overstated it**. “Two independent methods agree” held across all 26 sites but collapsed within either region (rho 0.33, p 0.27 for Indian steel alone). The agreement was the between-region contrast, not agreement on source strength. What rescued it: a logistic regression showing the 'is a steel plant' term survives every confound, stable at +0.31 to +0.36 — but is dwarfed by missing data (-0.99) and regional CO level (+0.58).
“What about the days when the satellite doesn't pick anything?”	133 of 468 site-days had been dropped silently — 28%, and highly uneven. Central Pacific lost 15 of 18; June and August lost half. This prompted the **within-site seasonal test** that became the strongest control in the project.

IN SUMMARY

What the two days produced

A dataset, a result, and — more valuable than either — a clear account of what the result can and cannot support.

The tangible output

- **335 tiles** across 26 sites and 18 days, evenly sampled, with CO, NO_2 and fire data for each.
- **A trained detector** and a score for every tile.
- **Nine scripts**, each tested, with an honest provenance file.
- **Eleven figures**, each one showing something the summary numbers hid.

The result

- Partial transfer: **AUC 0.77** for Indian steel against background, **0.54** for European — indistinguishable from chance.
- The detector's sensitivity to steel plants is **real but minor**, roughly a third the magnitude of its sensitivity to how much data is in the tile.
- A **within-site seasonal test**: Indian steel plants detect at 59% in the dry season and 21% in the monsoon ($p < 0.00001$), while background sites and European plants show no such swing. Same coordinates, same detector — only the weather changes.
- **Stacking works where single scenes fail**, with the negative control flat at zero.

What it means for the application

Schuit et al.'s second stage is a classifier over 41 engineered covariates — quality value, cloud fraction, albedo, surface pressure. Those are exactly the quantities that would absorb the confounds measured here and leave the plume term standing. **You independently derived the requirement for that stage, with coefficients attached, in two days.**

The honest closing note.

This demonstration is not the strongest thing in your application and was never meant to be. What carries it is the combination nobody else has: five years measuring carbon monoxide by hand, biomass-burning work in the region where CO contamination is worst, and Indian steel — the world's second-largest producer, and the place where a global catalogue will be hardest to trust.

Two days of code removed the one thing standing in front of all of that.